



Taller de Docker- IIS

Entregables:

1. Motores DB docker
2. Bases de datos del proyecto en docker
3. Backend y Frontend IIS
4. Backend y Frontend en Docker apuntando a la base de datos Docker

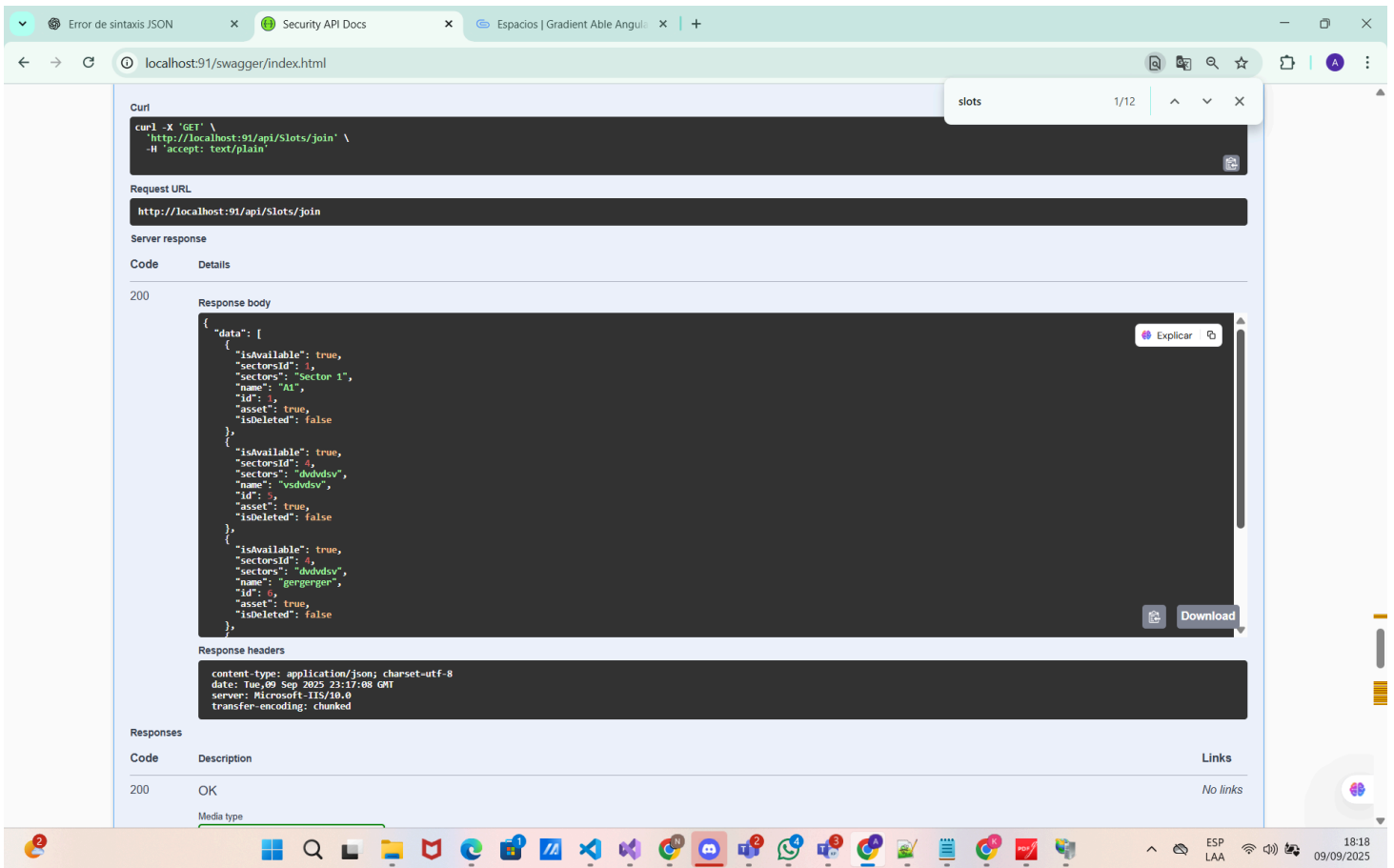
1. Backend y Frontend en IIS

En la imagen del Administrador de Internet Information Services (IIS) se observa que los sitios del proyecto fueron creados y están Iniciados con sus respectivos puertos y rutas físicas.

- Backend (AnprVision) desplegado correctamente en HTTP :91
- Frontend (AnprVisionFront) desplegado correctamente en HTTP :92

Nombre	Id.	Estado	Enlace	Ruta de acceso
AnprVision	3	Iniciado (h...	*:91 (http)	C:\inetpub\wwwroot\backIIS
AnprVisionFront	4	Iniciado (h...	*:92 (http)	C:\inetpub\wwwroot\front
Default Web Site	1	Iniciado (h...	*:80 (http)	%SystemDrive%\inetpub\wwwroot
sded	2	Iniciado (h...	*:90 (http)	C:\inetpub\wwwroot\back

Evidencias del back y front con sus respectivos puertos



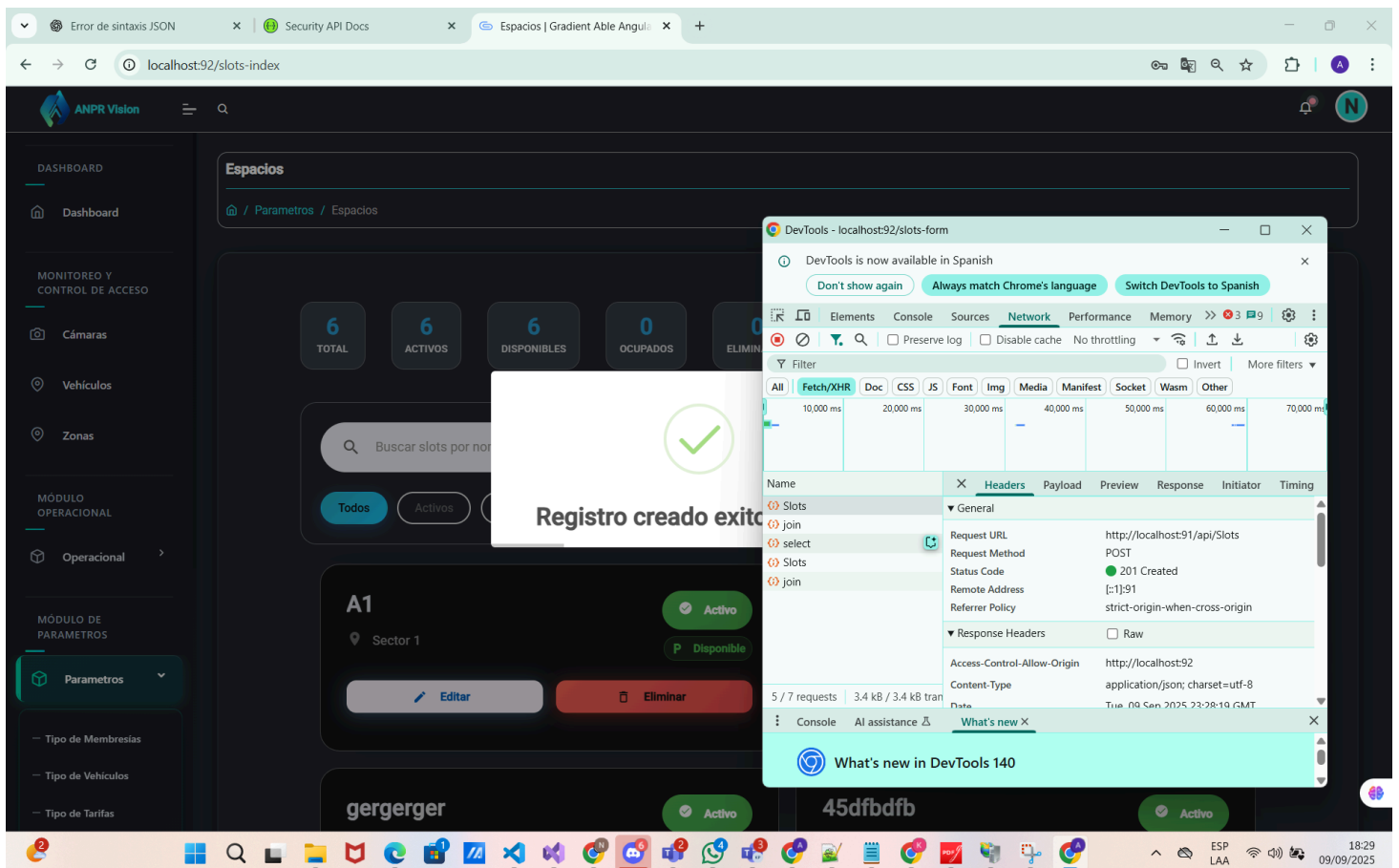
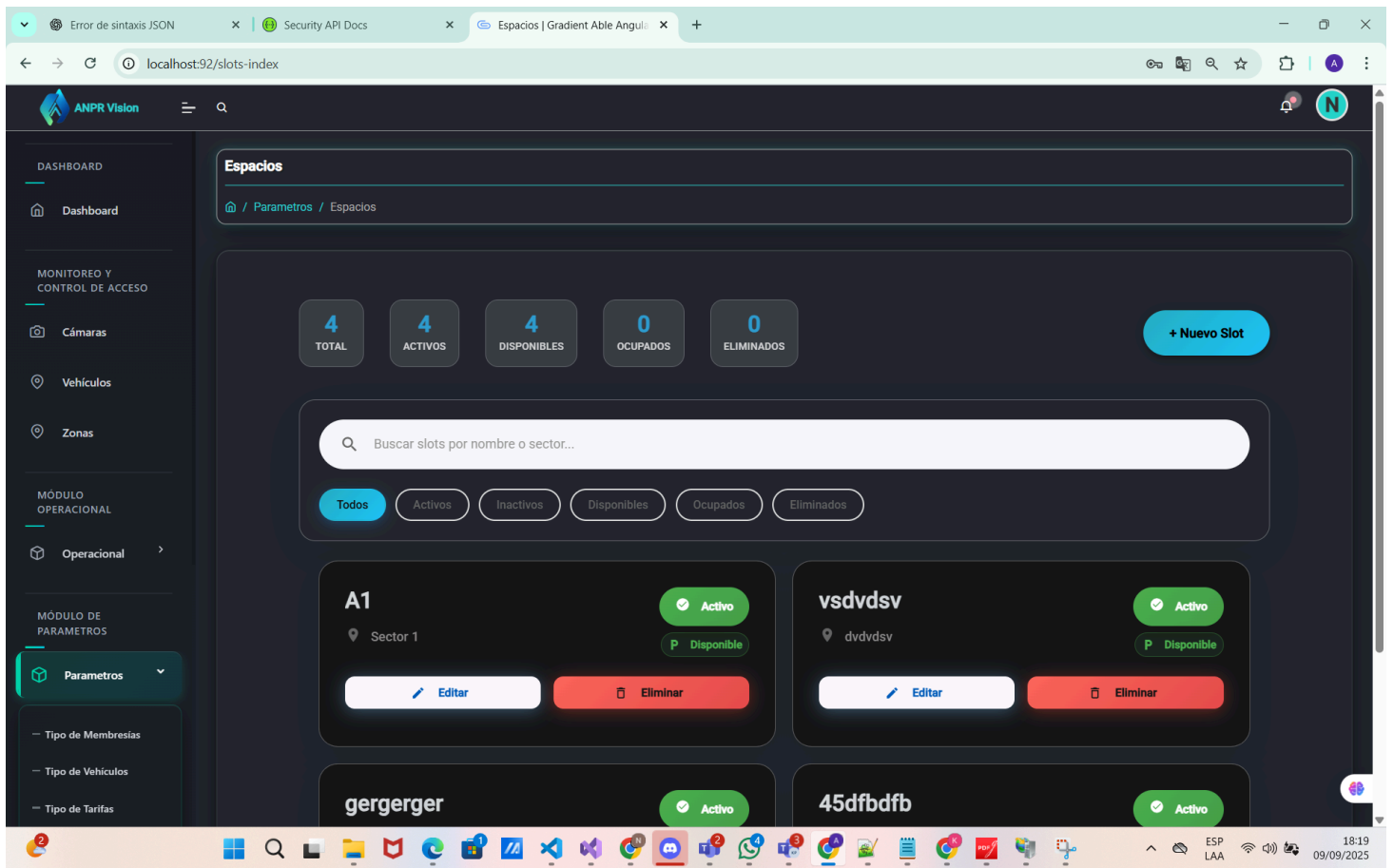
The screenshot shows a web browser window with the address bar displaying `localhost:91/swagger/index.html`. The browser has several tabs open, including "Error de sintaxis JSON", "Security API Docs", and "Espacios | Gradient Able Angul...".

The main content area displays the Swagger API interface for the `slots` endpoint. The interface shows the following details:

- Request URL:** `http://localhost:91/api/Slots/join`
- Server response:** 200
- Response body:** A JSON array containing three objects, each representing a slot. The objects have the following structure:

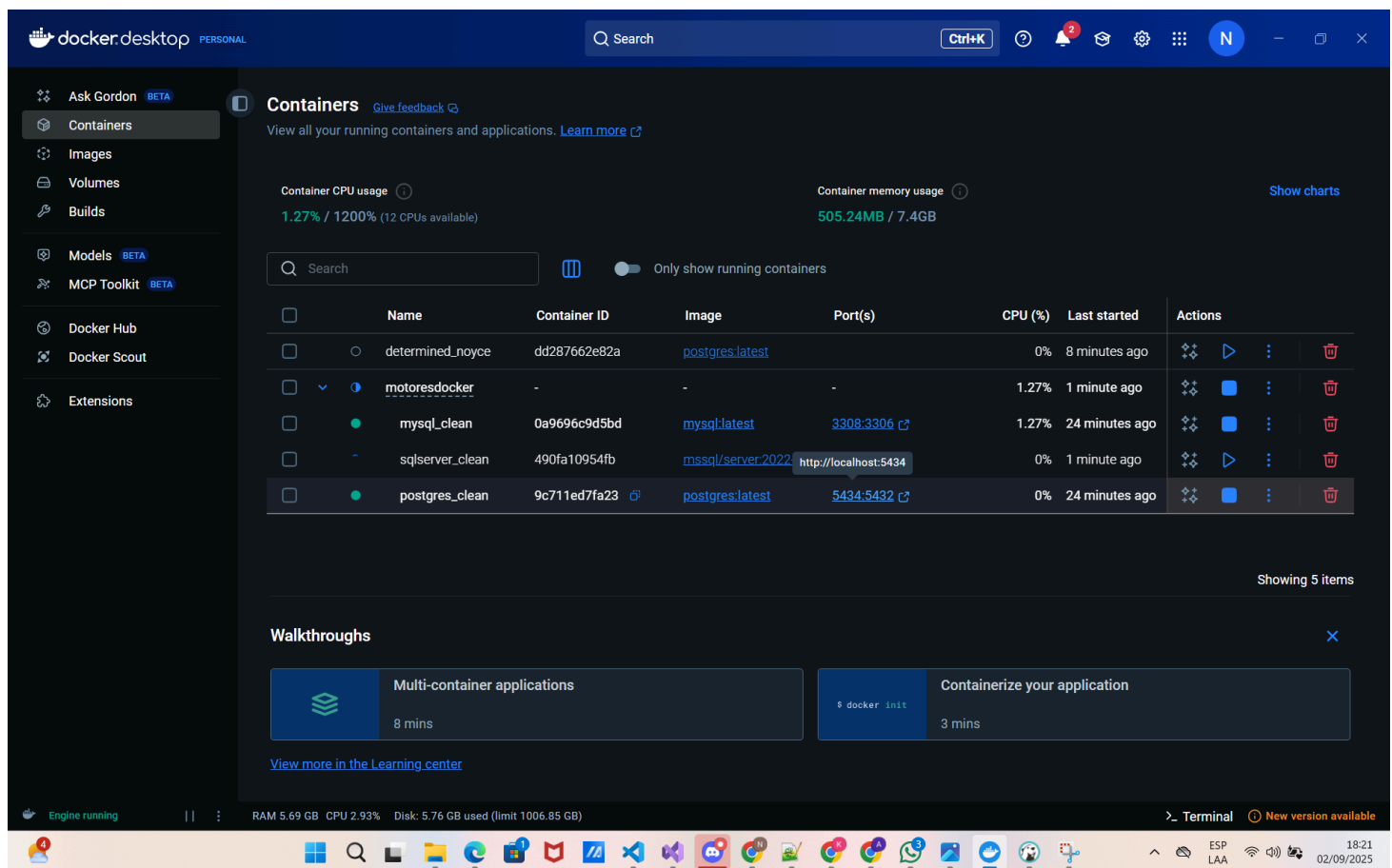
```
{  "data": [    {      "isAvailable": true,      "sectorId": 1,      "sectors": "sector 1",      "name": "A1",      "id": 1,      "asset": true,      "isDeleted": false    },    {      "isAvailable": true,      "sectorId": 4,      "sectors": "dvdvdsv",      "name": "ysdvdsv",      "id": 5,      "asset": true,      "isDeleted": false    },    {      "isAvailable": true,      "sectorId": 4,      "sectors": "dvdvdsv",      "name": "gergerger",      "id": 6,      "asset": true,      "isDeleted": false    }  ]}
```
- Response headers:** `content-type: application/json; charset=utf-8`, `date: Tue, 09 Sep 2025 23:17:08 GMT`, `server: Microsoft-IIS/10.0`, `transfer-encoding: chunked`
- Responses:** A table showing the response code (200) and description (OK).

The browser's taskbar at the bottom shows various application icons and the system clock indicating 18:18 on 09/09/2025.



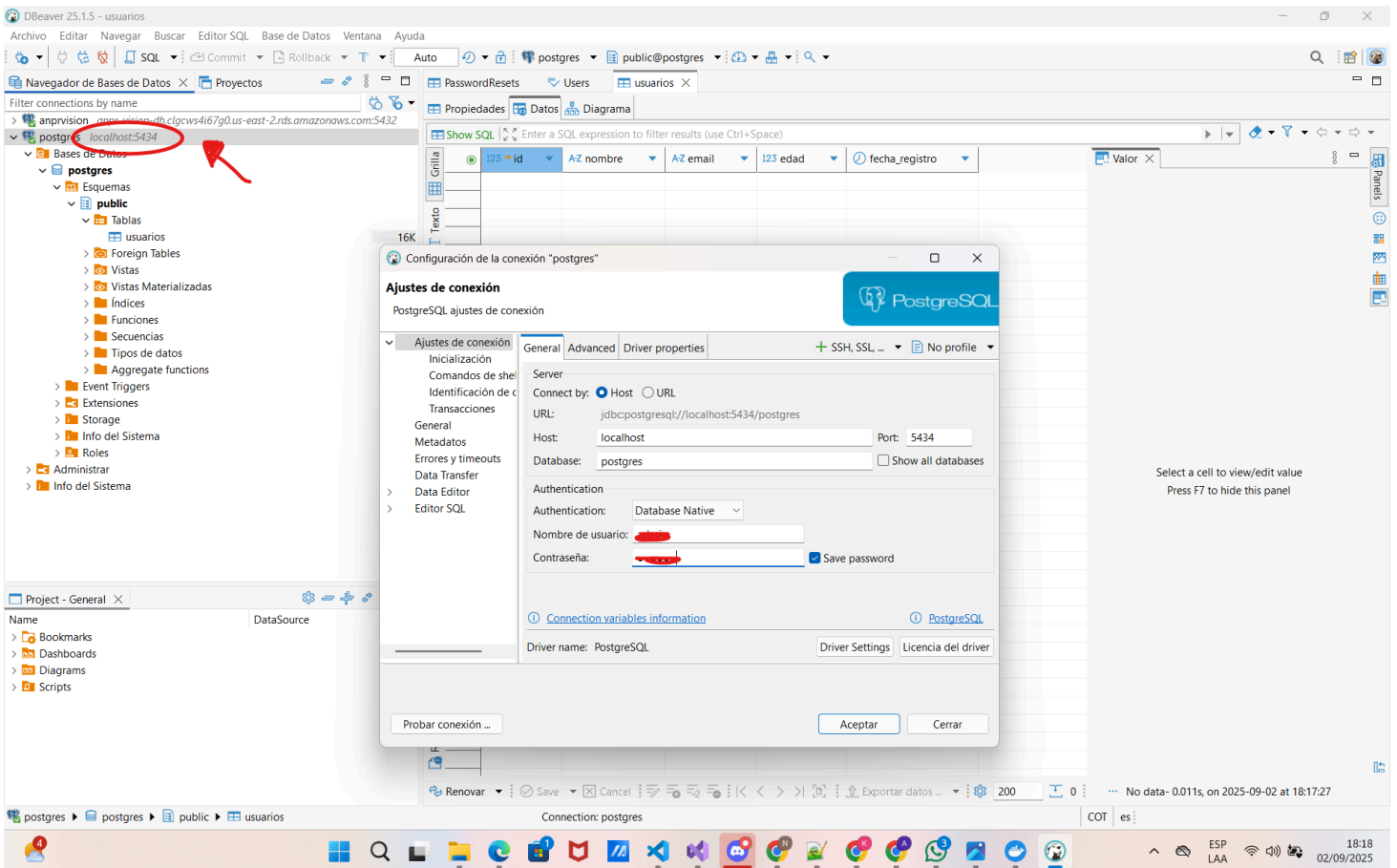
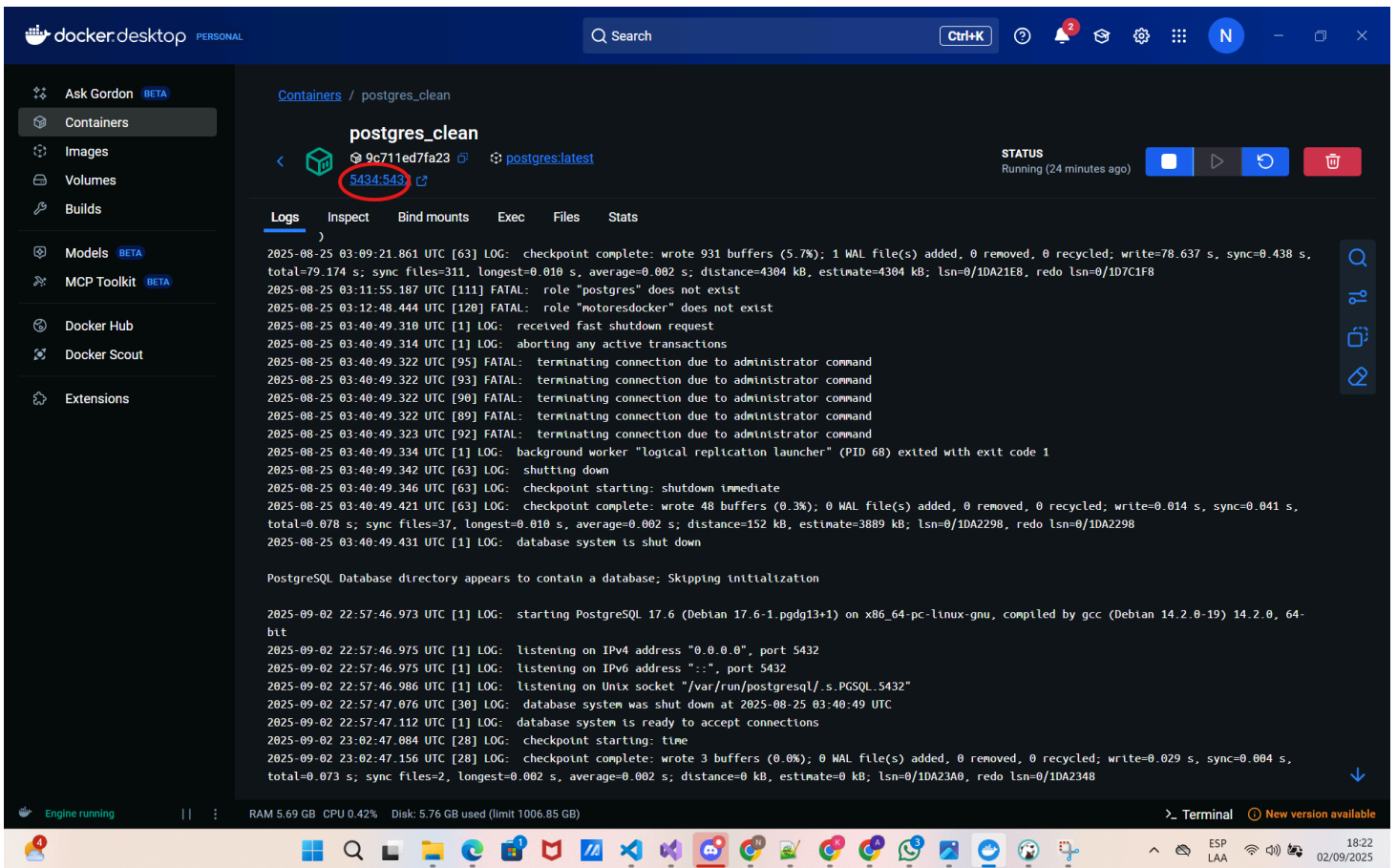
1. Motores de Bases de Datos en Docker

Vista general de contenedores en Docker Desktop: En la captura se ve el panel Containers de Docker Desktop con los motores de base de datos ejecutándose en contenedores separados. Esta vista confirma que los contenedores están arriba y listos para aceptar conexiones externas desde herramientas cliente o desde aplicaciones.



Conexión a PostgreSQL

La parte superior muestra los logs del contenedor de PostgreSQL indicando que el servidor se levantó correctamente y quedó a la espera de conexiones. Abajo se ve la conexión configurada en el cliente (DBeaver/pgAdmin) usando Host localhost, Puerto 5434, y las credenciales definidas en el contenedor. Esta combinación verifica que el mapeo de puertos es el correcto y que el motor en Docker responde como si fuera una instalación local, permitiendo explorar esquemas, tablas y ejecutar consultas sin salir del entorno de contenedores.



Conexión a MySQL

Ask Gordon

Containers

Images

Volumes

Builds

Models

MCP Toolkit

Docker Hub

Docker Scout

Extensions

Containers / mysql_clean

mysql_clean

0a9696c9d5bd

mysql:latest

3308:3306

STATUS

Running (36 minutes ago)

Logs

Inspect

Bind mounts

Exec

Files

Stats

2025-08-25T02:53:24.767783Z

0

[System]

[MY-011323]

[Server]

X Plugin ready for connections. Bind-address: '::' port: 33860, socket: /var/run/mysqld/mysqld.sock

2025-08-25T02:53:24.769207Z

0

[System]

[MY-010931]

[Server]

/usr/sbin/mysqld: ready for connections. Version: '9.4.0' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community Server - GPL.

2025-08-25T03:40:38.704259Z

0

[System]

[MY-013172]

[Server]

Received SHUTDOWN from user <via user signal>. Shutting down mysqld (Version: 9.4.0).

2025-08-25T03:40:40.403444Z

0

[System]

[MY-010910]

[Server]

/usr/sbin/mysqld: Shutdown complete (mysqld 9.4.0) MySQL Community Server - GPL.

2025-08-25T03:40:40.403513Z

0

[System]

[MY-015016]

[Server]

MySQL Server - end.

2025-09-02 22:57:46+00:00

0

[Note]

[Entrypoint]

Entrypoint script for MySQL Server 9.4.0-1.el9 started.

2025-09-02 22:57:47+00:00

0

[Note]

[Entrypoint]

Switching to dedicated user 'mysql'

2025-09-02 22:57:47+00:00

0

[Note]

[Entrypoint]

Entrypoint script for MySQL Server 9.4.0-1.el9 started.

2025-09-02T22:57:48.167182Z

0

[System]

[MY-015015]

[Server]

MySQL Server - start.

2025-09-02T22:57:48.886623Z

0

[System]

[MY-010116]

[Server]

/usr/sbin/mysqld (mysqld 9.4.0) starting as process 1

2025-09-02T22:57:48.886772Z

0

[System]

[MY-015590]

[Server]

MySQL Server has access to 12 logical CPUs.

2025-09-02T22:57:48.886821Z

0

[System]

[MY-015590]

[Server]

MySQL Server has access to 8131940352 bytes of physical memory.

2025-09-02T22:57:48.939466Z

1

[System]

[MY-013576]

[InnoDB]

InnoDB initialization has started.

2025-09-02T22:57:50.023030Z

1

[System]

[MY-013577]

[InnoDB]

InnoDB initialization has ended.

2025-09-02T22:58:40.278887Z

0

[Warning]

[MY-010060]

[Server]

CA certificate ca.pem is self signed.

2025-09-02T22:58:40.278888Z

0

[System]

[MY-013602]

[Server]

Channel mysql_native configured to support TLS. Encrypted connections are now supported for this channel.

2025-09-02T22:58:40.278895Z

0

[Warning]

[MY-011810]

[Server]

Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different directory.

2025-09-02T22:58:40.278900Z

0

[System]

[MY-011323]

[Server]

X Plugin ready for connections. Bind-address: '::' port: 33860, socket: /var/run/mysqld/mysqld.sock

2025-09-02T22:58:40.278901Z

0

[System]

[MY-010931]

[Server]

/usr/sbin/mysqld: ready for connections. Version: '9.4.0' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community Server - GPL.

2025-09-02T23:28:57.257321Z

11

[Warning]

[MY-013360]

[Server]

Plugin sha256_password reported: 'sha256_password' is deprecated and will be removed in a future release. Please use caching_sha2_password instead'

2025-09-02T23:30:02.643381Z

12

[Warning]

[MY-013360]

[Server]

Plugin sha256_password reported: 'sha256_password' is deprecated and will be removed in a future release. Please use caching_sha2_password instead'

2025-09-02T23:30:47.620402Z

13

[Warning]

[MY-013360]

[Server]

Plugin sha256_password reported: 'sha256_password' is deprecated and will be removed in a future release. Please use caching_sha2_password instead'

Engine running

RAM 5.75 GB CPU 2.42% Disk: 5.76 GB used (limit 1006.85 GB)

Terminal

New version available

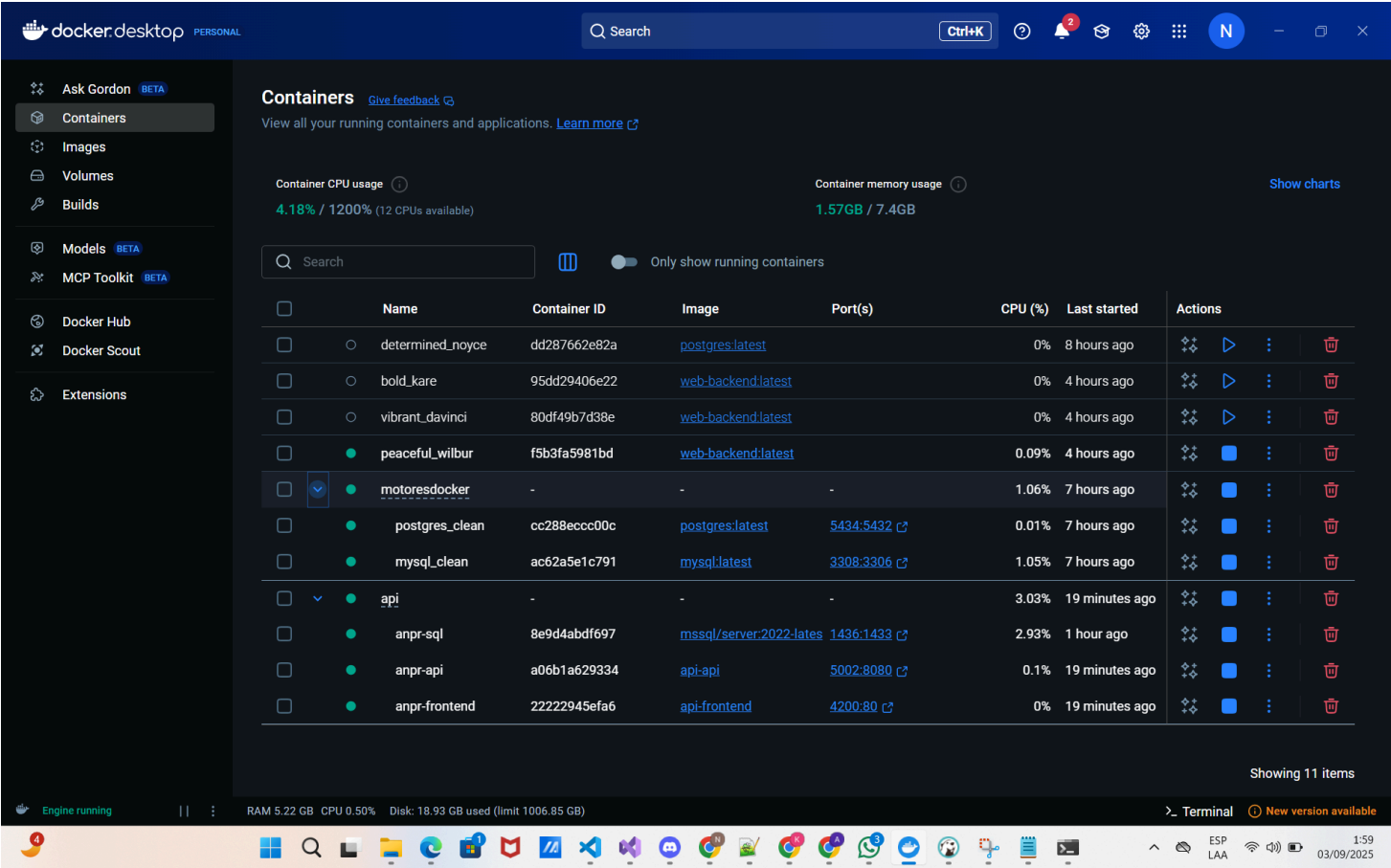
The screenshot shows the DBeaver 25.1.5 interface. The left sidebar displays the database tree with 'sys_config' selected. The main panel shows the table properties for 'sys_config', including columns: variable (varchar(128)), value (varchar(128)), set_time (timestamp), and set_by (varchar(128)). The bottom status bar indicates the table is 'sys_config'.

The screenshot shows the Docker Desktop interface. On the left is a sidebar with navigation options: Ask Gordon (BETA), Containers (selected), Images, Volumes, Builds, Models (BETA), MCP Toolkit (BETA), Docker Hub, Docker Scout, and Extensions. The main area displays the 'Containers' tab with a list of containers. The container 'sqlserver_clean' with ID '490fa10954fb' is selected. Its status is 'Running (50 seconds ago)'. Below the container list, the 'Logs' tab is active, showing a series of log entries from the container. The logs include messages about attribute synchronization, SQL Server Audit starting, SQL Trace ID 1 being started, server name, clearing tempdb, availability replica manager starting, an error about password validation, availability replica manager waiting, feature status, an error during server setup, SQL Trace being stopped, and starting up the 'msdb' database. The bottom of the window shows system information: Engine running, RAM 5.97 GB, CPU 11.98%, Disk 5.76 GB used (limit 1006.85 GB). The Windows taskbar is visible at the very bottom with various application icons.

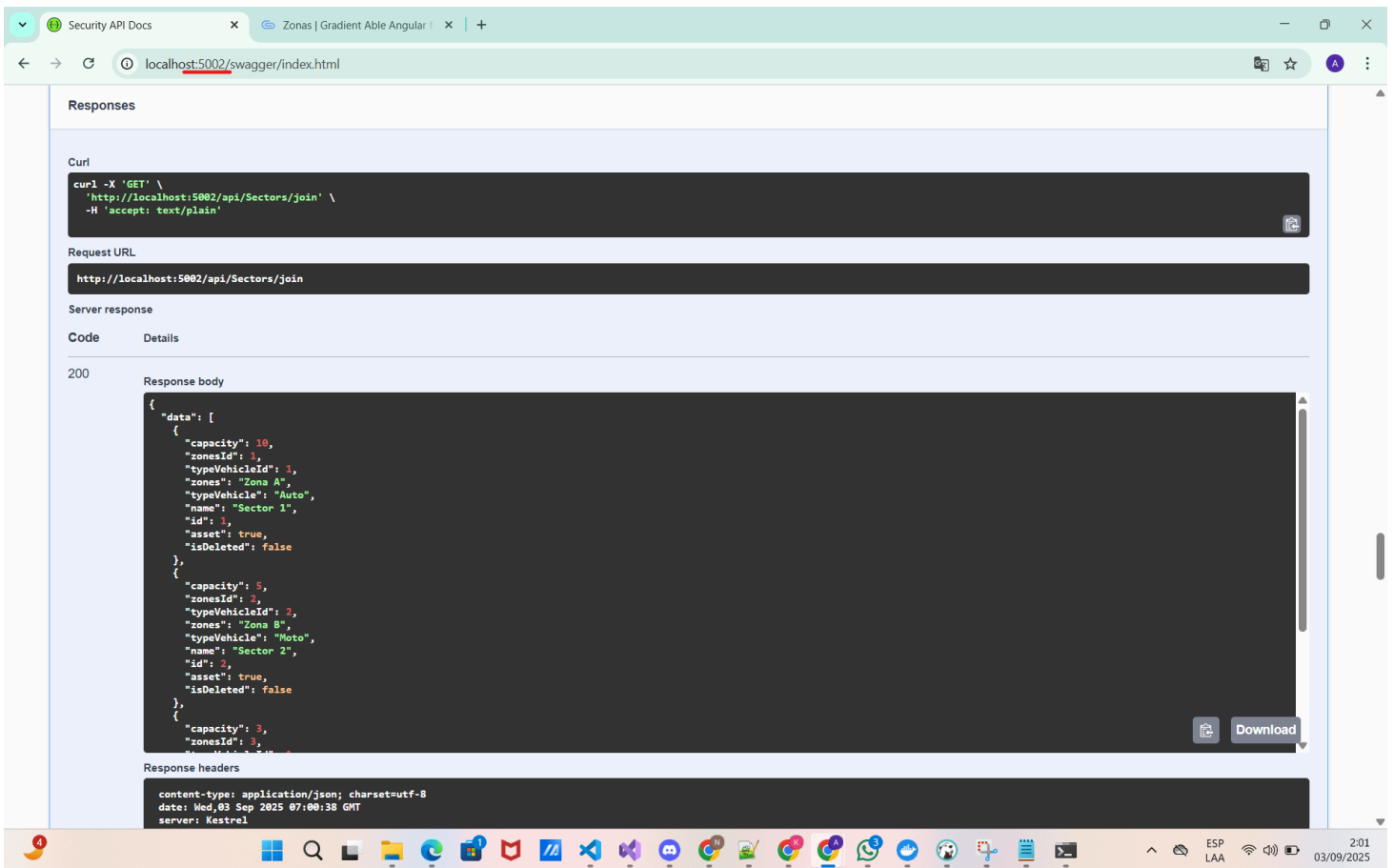
The screenshot shows the DBeaver 25.1.5 interface. In the left sidebar, the 'master' database is selected under the 'Databases' folder. The 'Properties' tab is active, showing the 'Name' as 'master' and the 'Database description' as '****'. The 'Schemas' tab is also visible, showing the 'dbo' schema. The bottom status bar indicates the current context is 'master'.

2. Backend y Frontend en Docker apuntando a la base de datos Docker

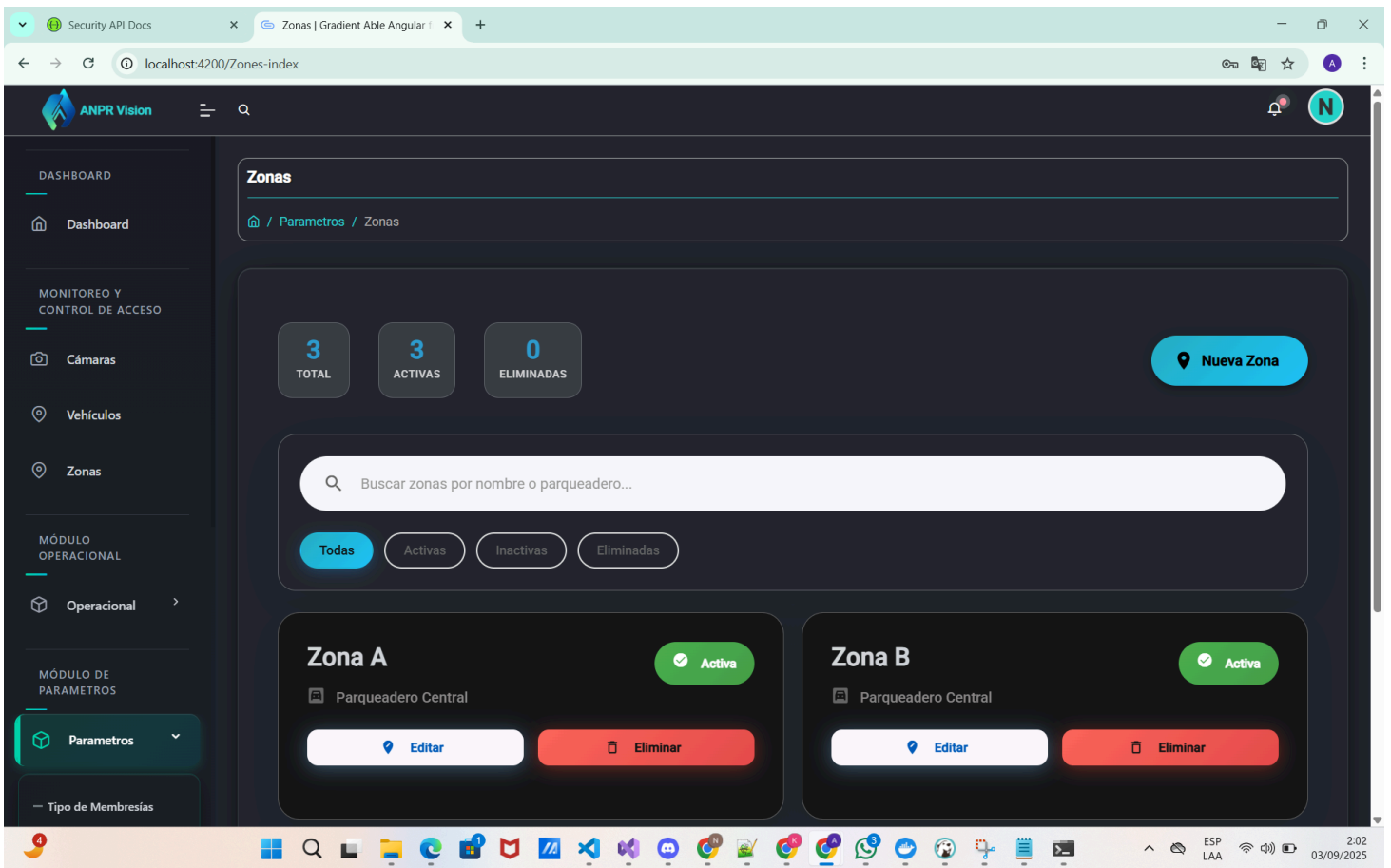
Vista general: contenedores de apps y motores de BD: La captura de Docker Desktop muestra ejecutándose, al mismo tiempo, los contenedores del backend, del frontend y de los motores de base de datos. Esta vista confirma que todos los servicios están arriba, comparten la misma red de Docker y pueden comunicarse por nombre de servicio.



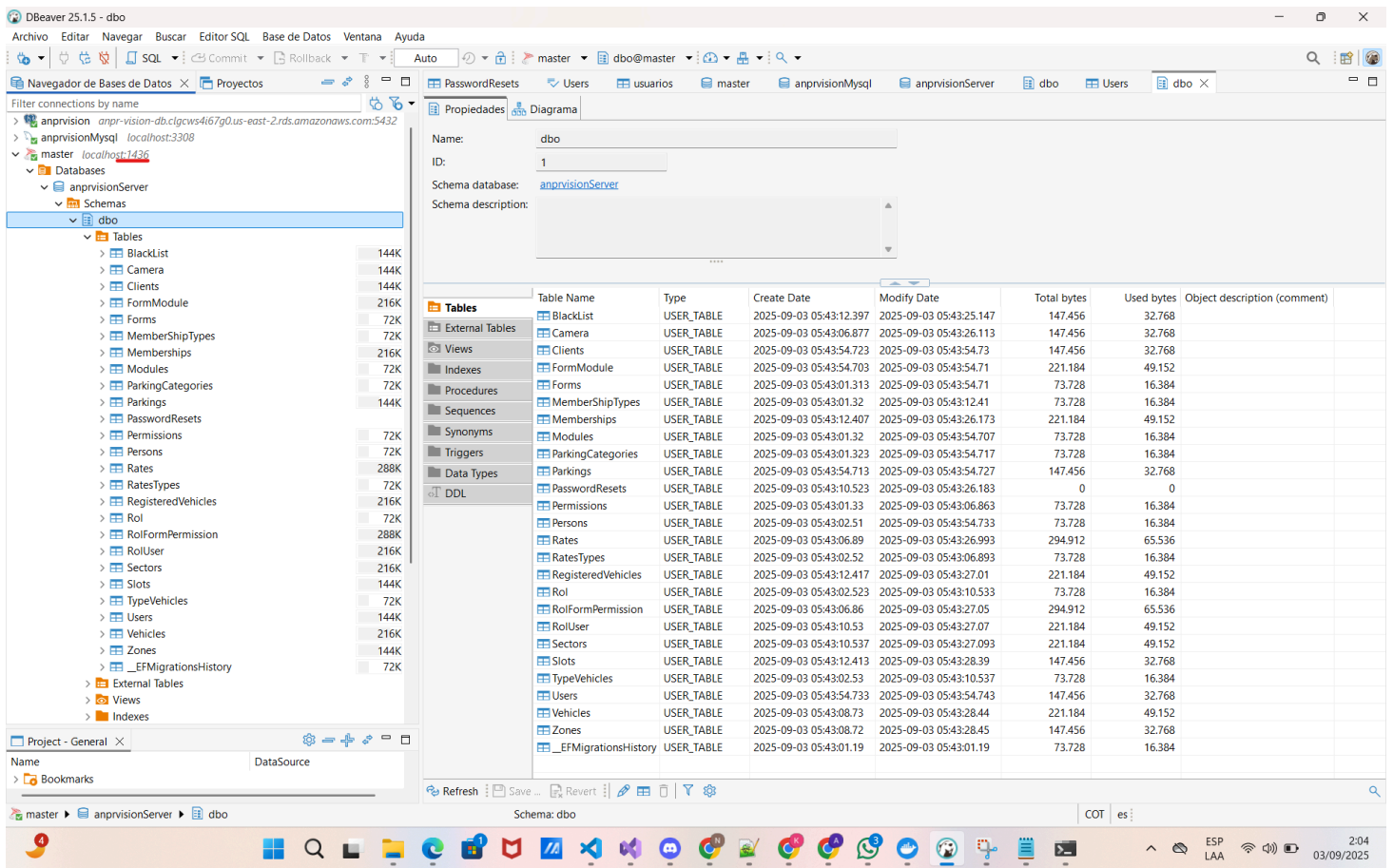
Prueba del backend con Swagger: En la sección inferior se abre Swagger UI de la API corriendo dentro del contenedor. Se ejecuta un endpoint (por ejemplo, /api/Sectors/join) y se observa el cURL, el Request URL y una respuesta 200 OK. Esto valida que el backend está correctamente expuesto hacia el host y que consulta la base de datos que también se ejecuta en Docker.



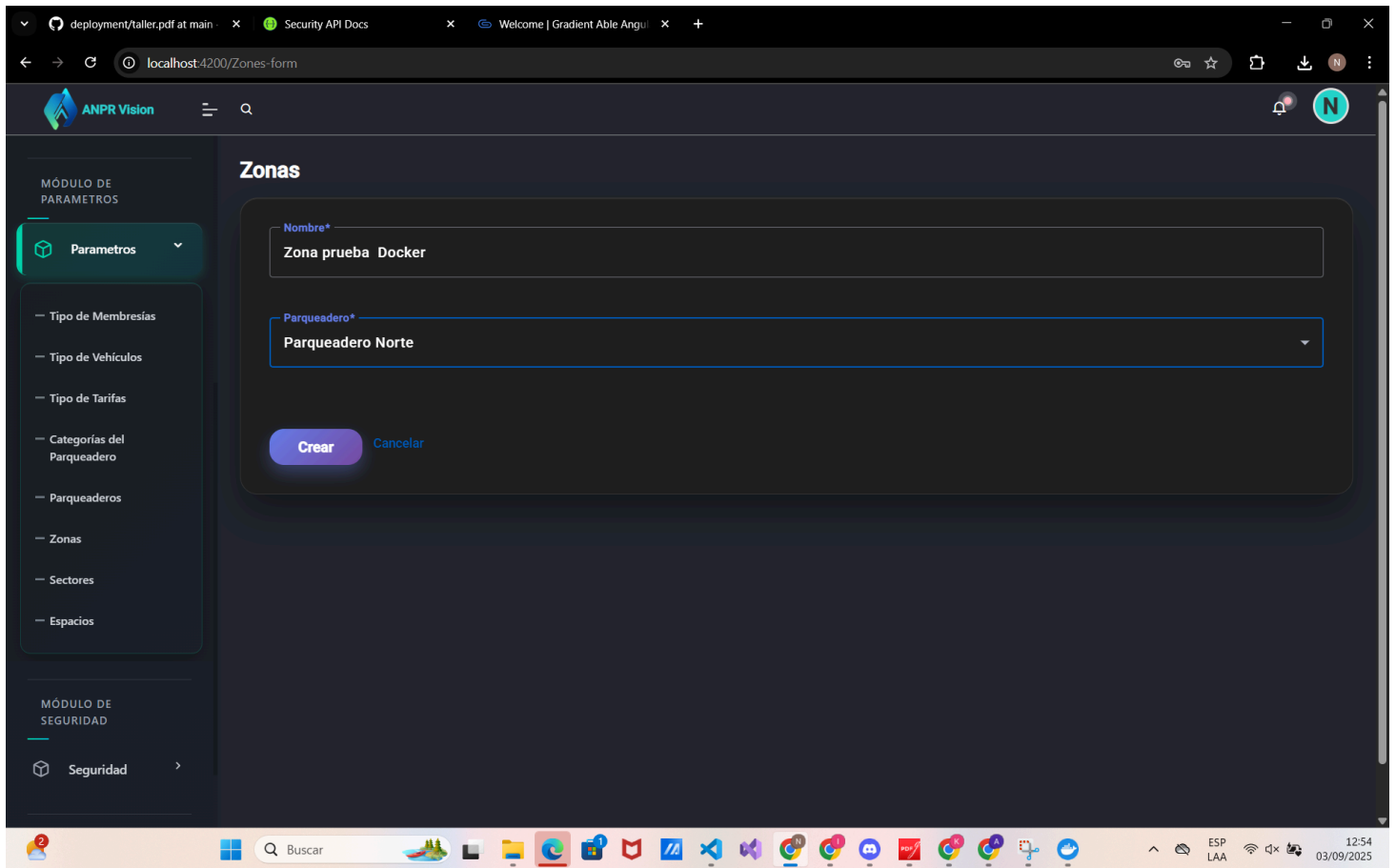
Frontend operativo y conectado al backend: La siguiente captura muestra el frontend en la pantalla de Zonas. Se visualizan tarjetas como Zona A y Zona B con sus estados, junto con filtros y acciones (Editar, Eliminar). Este resultado evidencia que el frontend está consumiendo la API y renderizando datos reales provenientes de la base de datos del entorno Docker.



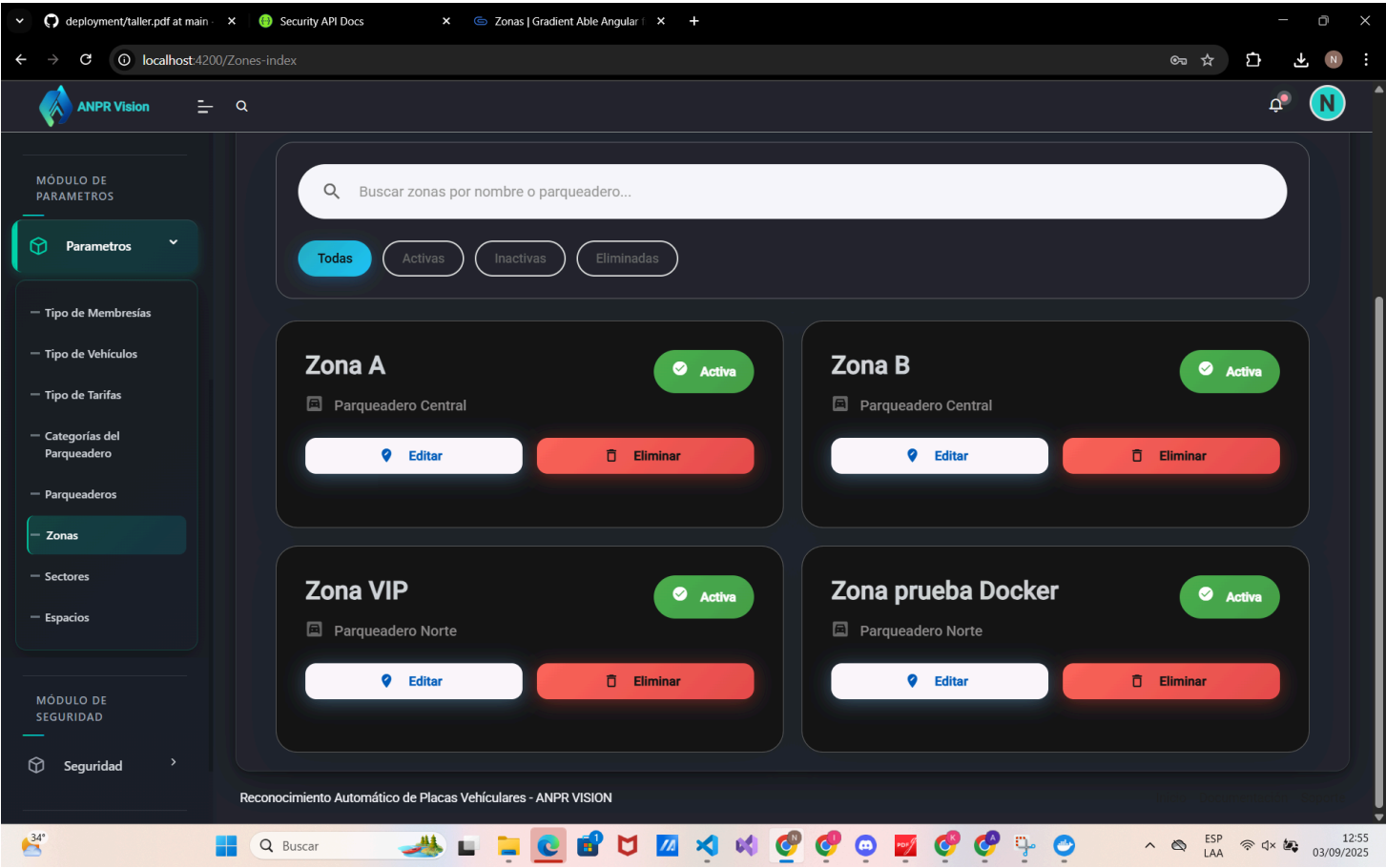
Verificación de datos en el cliente SQL: En la parte inferior se ve el cliente de base de datos (DBEaver) conectado al motor en Docker. Se inspeccionan esquemas y tablas del proyecto (por ejemplo, Zonas, Sectors, MembershipTypes, Rates, etc.). Esta comprobación cierra el circuito end-to-end: los datos mostrados en el frontend son los mismos que residen en la BD de Docker y que la API consulta en tiempo real.



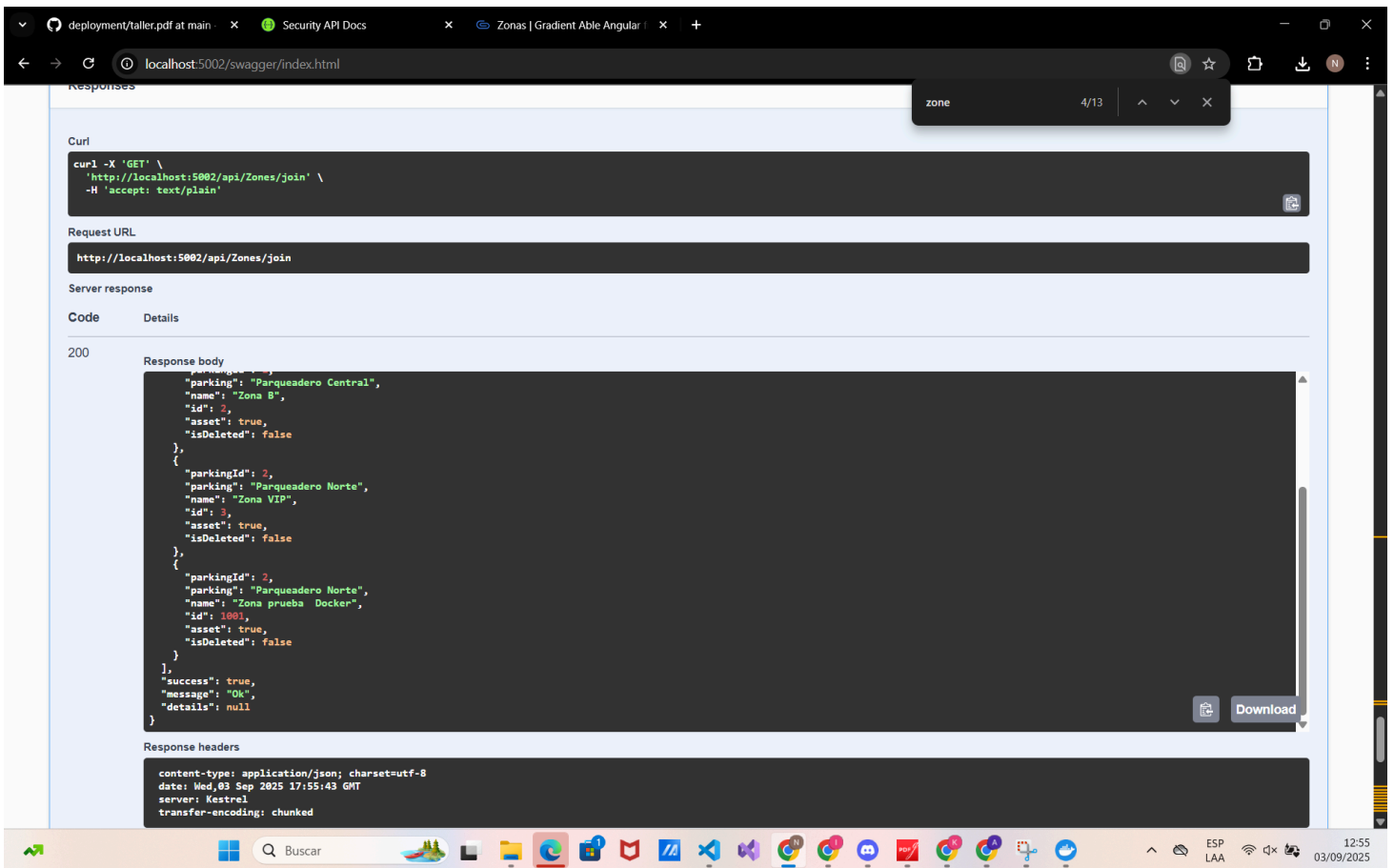
Realizamos un registro nuevo desde el fronted apuntando al backend montado en docker.



Registro creado



Verificamos que el registro se ha creado



Ubuntu

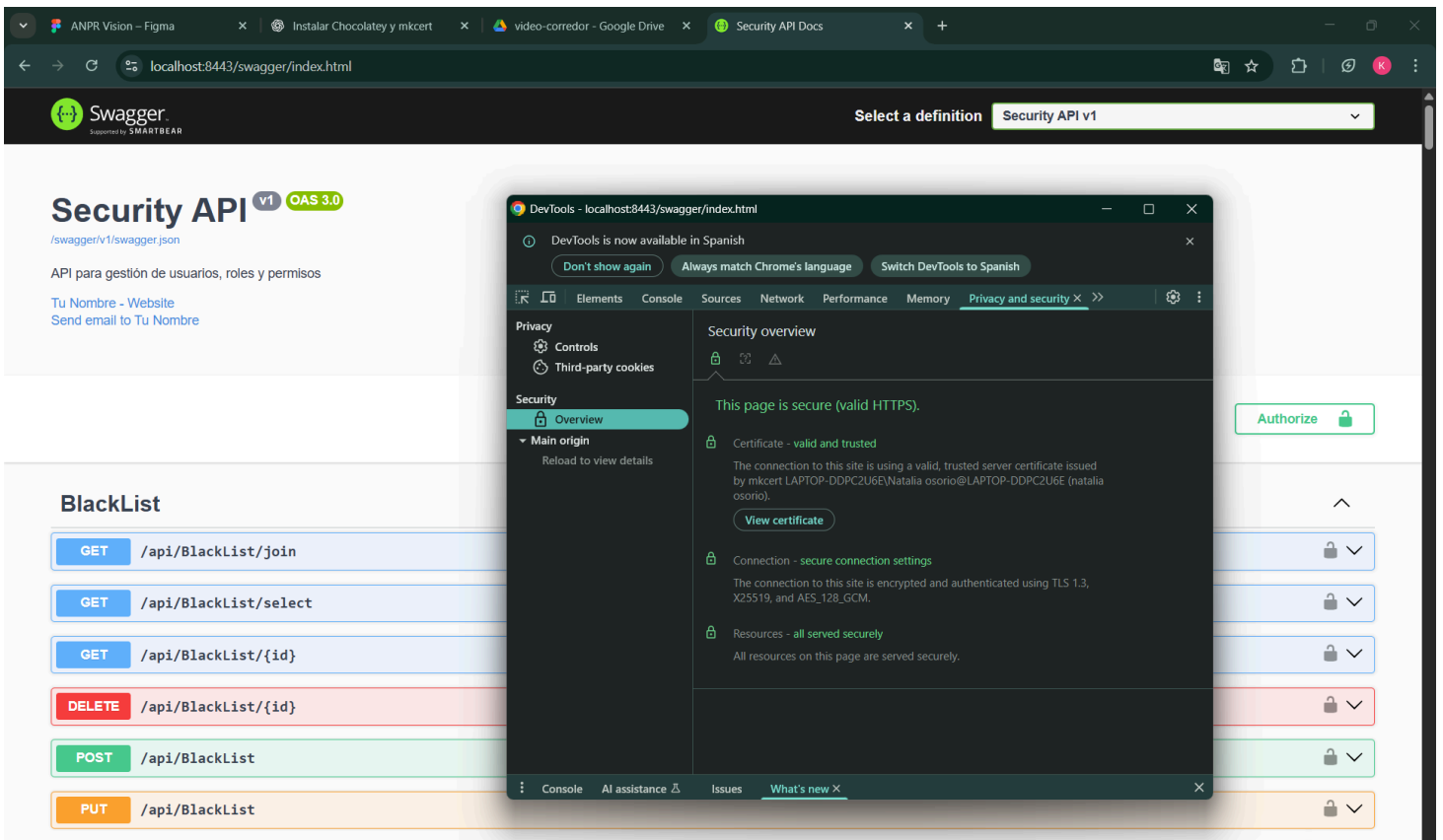
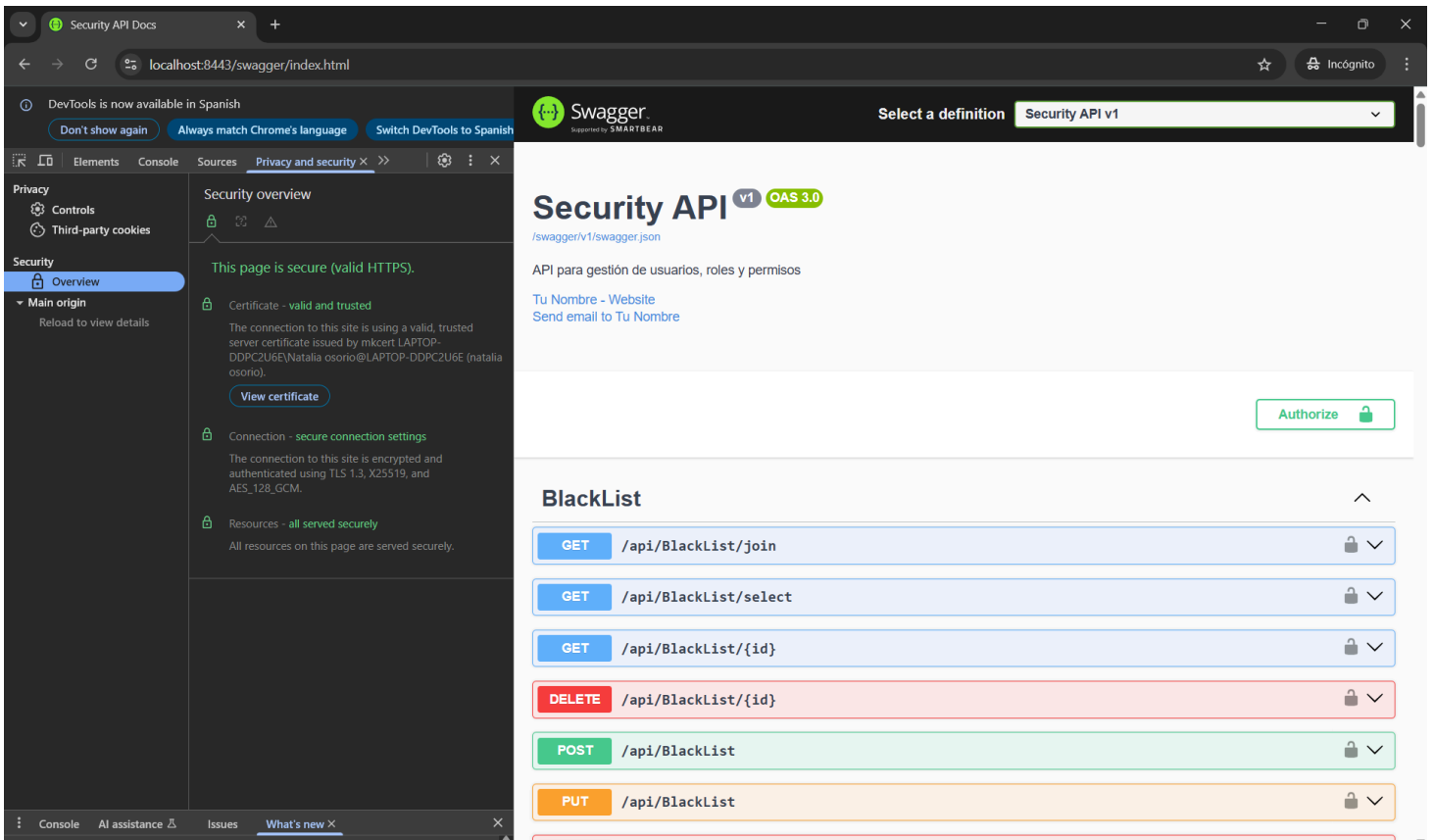
La imagen muestra la UI de Swagger en <https://localhost:8443/swagger/index.html> y, a la izquierda, el panel Security de DevTools indicando “This page is secure (valid HTTPS)”. Se ve que el certificado es válido y confiable, emitido por la CA de mkcert del equipo local. La sección Resources confirma que todos los recursos de la página se sirven de forma segura (sin contenido mixto).

Conclusión del proceso:

Se construyó un contenedor “todo en uno” (Ubuntu + SQL Server + [ASP.NET](#) Core + Nginx) con Kestrel escuchando en HTTP :5000 y Nginx haciendo la terminación TLS en :443 (expuesto como :8443 en el host).

Se resolvieron los problemas iniciales de arranque (elección del Web.dll y runtime .NET 9.0), se configuró la conexión a SQL Server interno (127.0.0.1,1433 con TrustServerCertificate/Encrypt=False), se añadió mkcert para un certificado confiable en local, se montó en Nginx y se ajustaron cabeceras de seguridad (HSTS y CSP: upgrade-insecure-requests).

El resultado final es una API servida por HTTPS válido en <https://localhost:8443>, con redirección limpia desde / a Swagger y sin advertencias de seguridad.



available